

Multi-Agent AI Recruitment Assessment System

DOCUMENT 3: Technology Stack Architecture & Engineering Rationale

Technology Selection Matrix & Architecture Overview

Every technology chosen in this platform was selected to optimize performance, modularity, reliability, data privacy, and explainability. The architecture adheres to enterprise SaaS best practices with zero vendor lock-in.

Domain	Technology Used	Engineering Rationale & Advantages
Backend Framework	FastAPI (Python 3.11)	Asynchronous high throughput, native OpenAPI documentation, automatic Pydantic request validation, and lightweight footprint.
Agent Workflow Engine	LangGraph (StateGraph)	Enables stateful multi-agent orchestration with conditional branching, cyclic retry loops, and deterministic execution.
LLM Gateway & Inference	Google Gemini 3.6 Flash / LiteLLM	Sub-second latency, native structured JSON mode, generous token context, and automatic fallback capabilities.
Speech-to-Text (STT)	OpenAI Whisper Architecture	High-accuracy acoustic transcription, automatic punctuation, timestamped diarization, and zero third-party audio leakage.
Code Execution Sandbox	Subprocess Hardened Sandbox	Safe evaluation of untrusted candidate code with 5s execution timeouts, memory bounds, and stdout/stderr capture.
Vector Semantic Search	Dense Vector Embeddings & Cosine Sim	Enables semantic skill matching that understands synonyms (e.g. 'PostgreSQL' matches 'Relational Databases') beyond keyword count.
Relational Database	SQLAlchemy ORM + SQLite	ACID-compliant storage, relational integrity, zero external DB dependencies for seamless local installation and testing.
Frontend Framework	React 18 + TypeScript + Vite	Type-safe component architecture, sub-second HMR development, production bundle optimization, and high responsiveness.
UI Styling & Theme	Tailwind CSS + Lucide Icons	Modern enterprise dark-mode design system, glassmorphic panels, animated status indicators, and clean typography.
Testing & Quality	Pytest + Pytest-Asyncio	Comprehensive test coverage validating all 6 agents and full multi-agent pipeline in under 6 seconds.

1. Why LangGraph for Multi-Agent Orchestration?

Traditional sequential LLM chains cannot gracefully handle branching logic (e.g. retrying code execution if a syntax error occurs, or skipping virtual interviews for senior candidates). LangGraph models the recruitment lifecycle as a state machine (StateGraph). Each agent updates a shared, immutable RecruitmentGraphState dictionary, allowing deterministic transitions, human-in-the-loop checkpoints, and full observability.

2. Why OpenAI Whisper for Speech-to-Text?

Recruitment voice screening requires high transcription accuracy across diverse accents, technical jargon (e.g., 'Kubernetes', 'FastAPI', 'Redis Streams'), and ambient noise. Whisper provides state-of-the-art word error rate (WER) and outputs precise timestamped segments, enabling the STAR Behavioral Agent to map exact candidate speech segments to Situation, Task, Action, and Result.

3. Why Demographic-Blinded Fair Scoring Architecture?

Unconscious recruiter bias is a major obstacle in modern talent acquisition. By decoupling candidate personal identifiers (Name, Gender, Age, Ethnicity, University Brand) from the evaluation matrix and assigning a deterministic weight formula (Skills 35%, Coding 35%, Interview 20%, Protocol 10%), the system guarantees that shortlisting decisions are 100% merit-based and legally auditable.

4. Why React + TypeScript + Vite for Frontend?

Recruiters and candidates need instant feedback. Vite delivers lightning-fast build times (under 3s), TypeScript guarantees 100% type safety across API response structures, and React's reactive state handles live WebSocket progress streaming seamlessly without full-page reloads.